

Example 15.1

Suppose a TCP connection is transferring a file of 5,000 bytes. The first byte is numbered 10,001. What are the sequence numbers for each segment if data are sent in five segments, each carrying 1,000 bytes?

Solution

The following shows the sequence number for each segment:

Segment 1	→	Sequence Number:	10,001	Range:	10,001	to	11,000
Segment 2	→	Sequence Number:	11,001	Range:	11,001	to	12,000
Segment 3	→	Sequence Number:	12,001	Range:	12,001	to	13,000
Segment 4	→	Sequence Number:	13,001	Range:	13,001	to	14,000
Segment 5	→	Sequence Number:	14,001	Range:	14,001	to	15,000

The value in the sequence number field of a segment defines the number assigned to the first data byte contained in that segment.

When a segment carries a combination of data and control information (piggy-backing), it uses a sequence number. If a segment does not carry user data, it does not logically define a sequence number. The field is there, but the value is not valid. However, some segments, when carrying only control information, need a sequence number to allow an acknowledgment from the receiver. These segments are used for connection establishment, termination, or abortion. Each of these segments consume one sequence number as though it carries one byte, but there are no actual data. We will elaborate on this issue when we discuss connections.

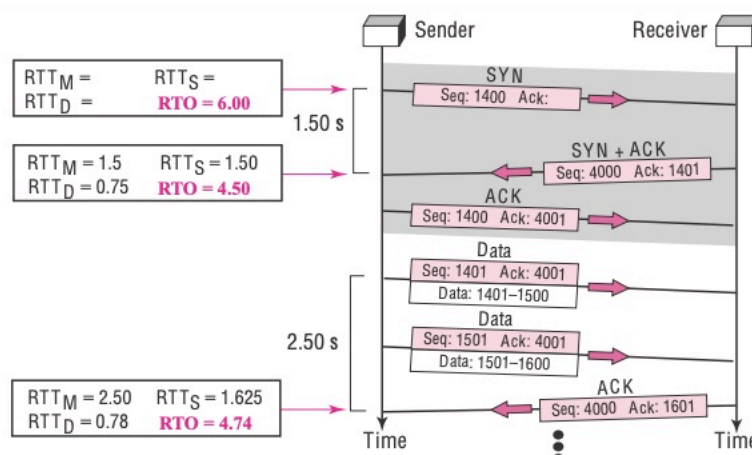
Example 15.3

Let us give a hypothetical example. Figure 15.39 shows part of a connection. The figure shows the connection establishment and part of the data transfer phases.

1. When the SYN segment is sent, there is no value for RTT_M , RTT_S , or RTT_D . The value of RTO is set to 6.00 seconds. The following shows the value of these variables at this moment:

$$RTO = 6$$

Figure 15.39 Example 15.3



2. When the SYN+ACK segment arrives, RTT_M is measured and is equal to 1.5 seconds. The following shows the values of these variables:

$$\begin{aligned} RTT_M &= 1.5 \\ RTT_S &= 1.5 \\ RTT_D &= (1.5) / 2 = 0.75 \\ RTO &= 1.5 + 4 \times 0.75 = 4.5 \end{aligned}$$

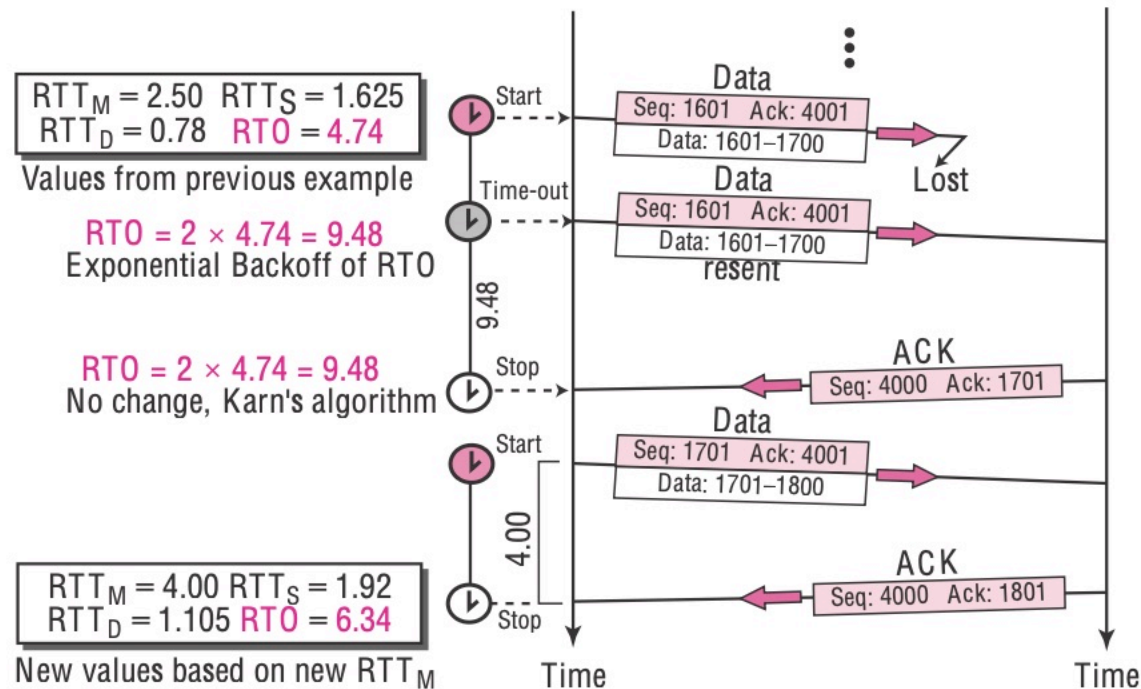
3. When the first data segment is sent, a new RTT measurement starts. Note that the sender does not start an RTT measurement when it sends the ACK segment, because it does not consume a sequence number and there is no time-out. No RTT measurement starts for the second data segment because a measurement is already in progress. The arrival of the last ACK segment is used to calculate the next value of RTT_M . Although the last ACK segment acknowledges both data segments (accumulative), its arrival finalizes the value of RTT_M for the first segment. The values of these variables are now as shown below.

$$\begin{aligned} RTT_M &= 2.5 \\ RTT_S &= 7/8 \times 1.5 + (1/8) \times 2.5 = 1.625 \\ RTT_D &= 3/4 (7.5) + (1/4) \times |1.625 - 2.5| = 0.78 \\ RTO &= 1.625 + 4 \times 0.78 = 4.74 \end{aligned}$$

Example 15.4

Figure 15.40 is a continuation of the previous example. There is retransmission and Karn's algorithm is applied.

Figure 15.40 Example 15.4

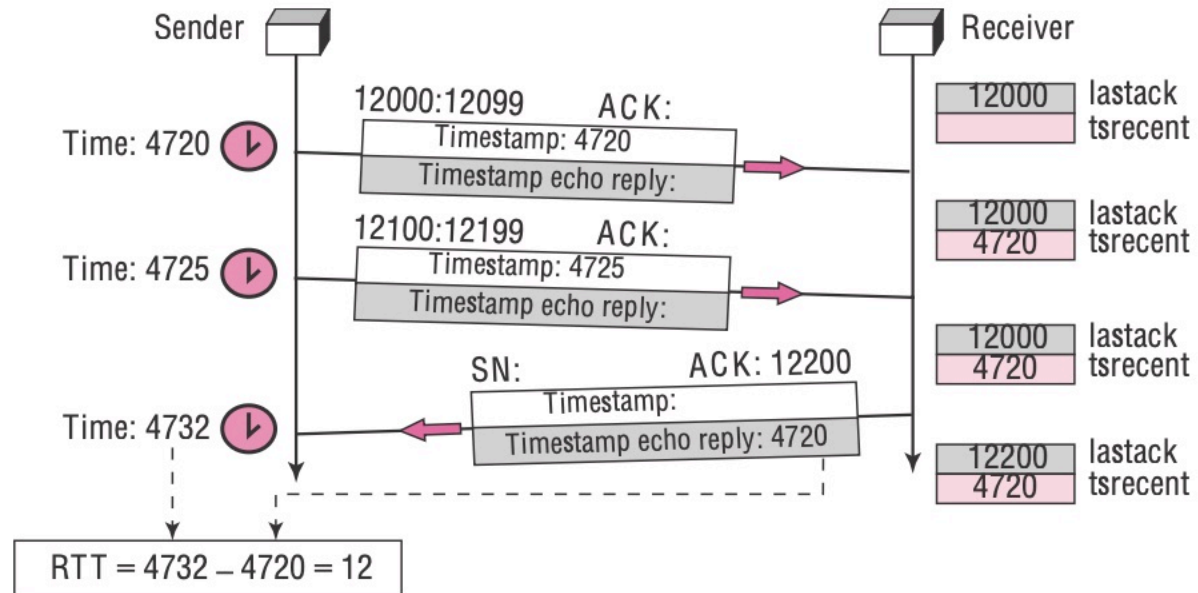


The first segment in the figure is sent, but lost. The RTO timer expires after 4.74 seconds. The segment is retransmitted and the timer is set to 9.48, twice the previous value of RTO. This time an ACK is received before the time-out. We wait until we send a new segment and receive the ACK for it before recalculating the RTO (Karn's algorithm).

Example 15.5

Figure 15.47 shows an example that calculates the round-trip time for one end. Everything must be flipped if we want to calculate the RTT for the other end.

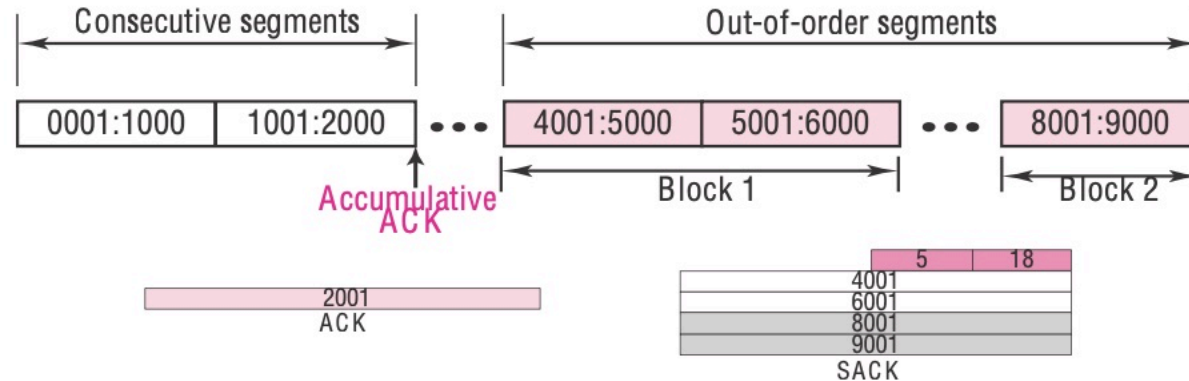
Figure 15.47 *Example 15.5*



Example 15.6

Let us see how the SACK option is used to list out-of-order blocks. In Figure 15.49 an end has received five segments of data.

Figure 15.49 Example 15.6

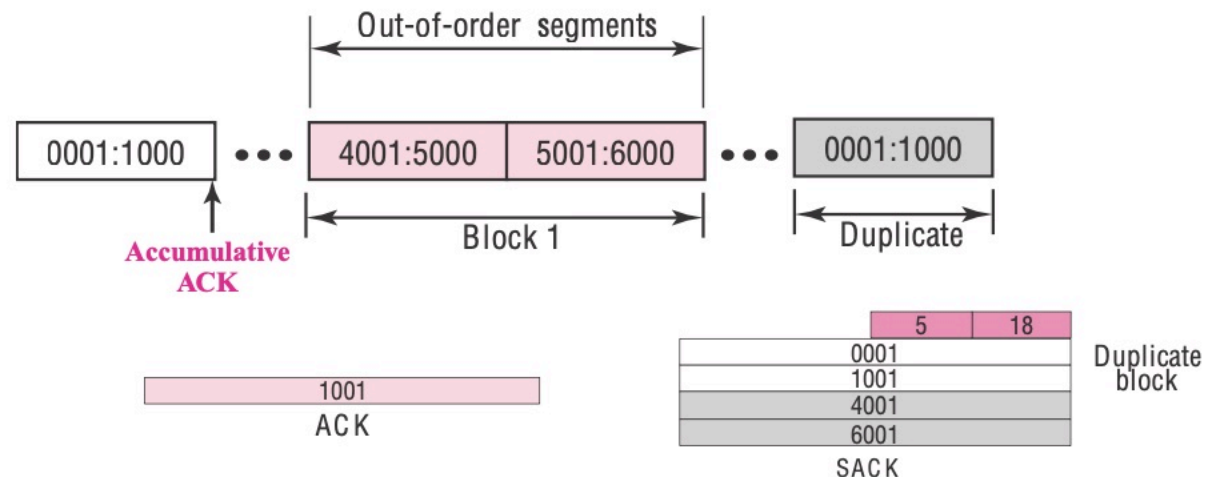


The first and second segments are in consecutive order. An accumulative acknowledgment can be sent to report the reception of these two segments. Segments 3, 4, and 5, however, are out of order with a gap between the second and third and a gap between the fourth and the fifth. An ACK and a SACK together can easily clear the situation for the sender. The value of ACK is 2001, which means that the sender need not worry about bytes 1 to 2000. The SACK has two blocks. The first block announces that bytes 4001 to 6000 have arrived out of order. The second block shows that bytes 8001 to 9000 have also arrived out of order. This means that bytes 2001 to 4000 and bytes 6001 to 8000 are lost or discarded. The sender can resend only these bytes.

Example 15.7

Figure 15.50 shows how a duplicate segment can be detected with a combination of ACK and SACK. In this case, we have some out-of-order segments (in one block) and one duplicate segment. To show both out-of-order and duplicate data, SACK uses the first block, in this case, to show the duplicate data and other blocks to show out-of-order data. Note that only the first block can be used for duplicate data. The natural question is how the sender, when it receives these ACK and SACK values, knows that the first block is for duplicate data (compare this example with the previous example). The answer is that the bytes in the first block are already acknowledged in the ACK field; therefore, this block must be a duplicate.

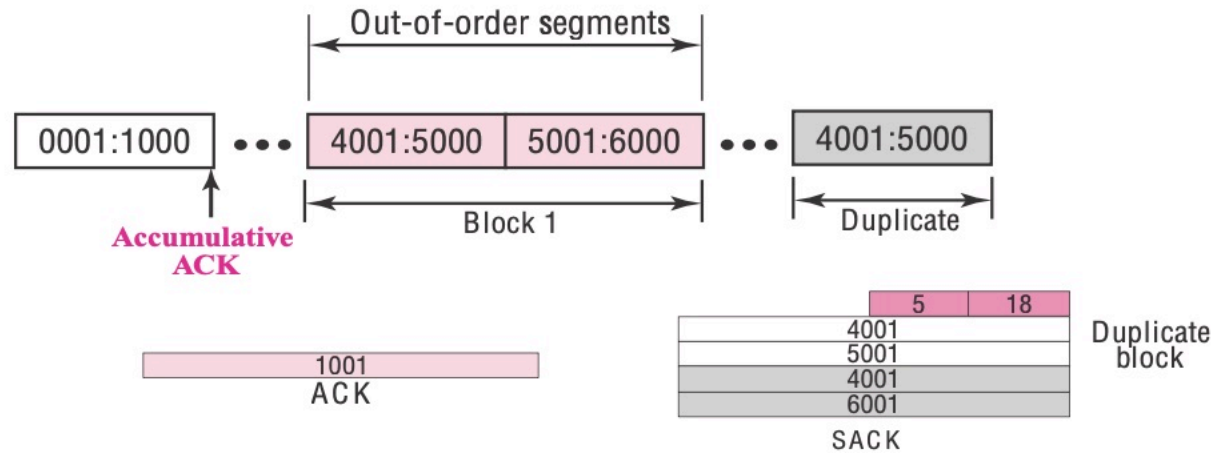
Figure 15.50 Example 15.7



Example 15.8

Figure 15.51 shows what happens if one of the segments in the out-of-order section is also duplicated. In this example, one of the segments (4001:5000) is duplicated.

Figure 15.51 *Example 15.8*



The SACK option announces this duplicate data first and then the out-of-order block. This time, however, the duplicated block is not yet acknowledged by ACK, but because it is part of the out-of-order block (4001:5000 is part of 4001:6000), it is understood by the sender that it defines the duplicate data.